The developers effectively provide a library of reusable components and methods to be used by steps in the Business Flow layer. For example, if you're working with a web application, you'll design *page objects*—classes that hide the technical details about HTML fields and CSS classes behind descriptively named methods, such as `enterName()` or `submitApplicationRequest()`.

Following on from the previous example, suppose you were writing a Technical layer using Thucydides and WebDriver, a popular open source web automation tool that we'll discuss in detail in chapter 8. Using this technology stack, the `selectMenuOptions()` method might look like this:

```
public class HomePage extends PageObject {

    public void selectMenuOptions(String... menuOptions) {
        for(String menuChoice : menuOptions) {
            $(".mainMenu").findBy(By.linkText(menuChoice)).click();
        }
    }
}
```

A nice readable API method.

The nitty-gritty CSS and WebDriver logic are hidden away.

A well-written Technical layer does wonders to isolate the other layers from the impact of low-level changes. For example, suppose the design of the registration page changes, involving changes to the HTML structure and field names. Such a change would modify neither the business rule nor the workflow for this requirement, and those levels wouldn't be affected. The only code you'd need to update is within the page object that encapsulates the registration page. This update would work for any scenario that uses this page.

In addition, you can implement the Business Rules and the Business Flow layers *before* the user interface has been implemented. The first layers are written in collaboration with the testers and business analysts and act as guidelines for development work. Only when the user interface is reasonably stable do you implement the technical components.

7.3.4 How many layers?

The precise number of layers you need to make your automated tests readable and maintainable will vary depending on the scope and complexity of the acceptance criteria. There are many different types of acceptance criteria, and some will be more complicated and involved than others.

Some will describe how a user interacts with the application to achieve some business goal, narrating the user's journey through the system and the expected business outcomes, like the "Registering online for a new Frequent Flyer account" scenario you saw earlier. Others will focus on more specific business rules, as with the Frequent Flyer point calculation examples you saw in chapter 6:

```
Scenario: Calculating travel and status points for a domestic flight
Given the flying distance between Sydney and Melbourne is 878 km
And I am a standard Frequent Flyer member
```